Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

12. Triển khai & DevOps . . . 2
12.1. Thách thức triển khai microservices . . . 3
12.1.1. Vấn đề: từ "triển khai một tệp WAR" đến "điều phối N service độc lập" . . . 3
12.1.2. Từ thủ công đến tự động hóa — tư duy DevOps . . . 3
12.2. Containerization với Docker . . . 4
12.2.1. Vấn đề: "Works on my machine" . . . 4
12.2.2. Container — cô lập gọn nhẹ . . . 4
12.2.3. Các mô hình triển khai — từ đặc thù ngôn ngữ đến Serverless . . . 5
12.2.4. Dockerfile — định nghĩa container image . . . 6
12.3. Docker Compose — điều phối trên một máy . . . 7
12.3.1. Vấn đề: khởi chạy thủ công nhiều dịch vụ, cơ sở dữ liệu và Kafka . . . 7
12.3.2. Docker Compose — khai báo nhiều service . . . 7
12.3.3. Docker Compose — điểm mạnh và giới hạn . . . 9
12.4. Pipeline CI/CD cho microservices . . . 9
12.4.1. Vấn đề: N service × M giai đoạn = độ phức tạp nhân bội . . . 9
12.4.2. Kiến trúc CI/CD cho microservices . . . 10
12.4.3. Mono-repo và Poly-repo — ảnh hưởng đến CI/CD . . . 10
12.4.4. Tối ưu hóa pipeline . . . 11
12.4.5. Ví dụ thực tế: pipeline CI/CD cho KBM . . . 11
12.4.6. Quản lý secret . . . 14
12.4.7. Database Migration trong pipeline CI/CD . . . 15
12.5. Các chiến lược triển khai . . . 16
12.5.1. Vấn đề: triển khai phiên bản mới mà không gây gián đoạn . . . 16
12.5.2. Rolling Update . . . 16
12.5.3. Blue/Green Deployment . . . 17
12.5.4. Canary Release . . . 17
12.5.5. Shadow Deployment . . . 18
12.5.6. So sánh . . . 18
12.6. Infrastructure as Code và nền tảng điều phối triển khai . . . 19
12.6.1. Vấn đề: "Ai cấu hình máy chủ? Cấu hình nằm ở đâu?" . . . 19
12.6.2. IaC — hạ tầng dưới dạng mã nguồn . . . 19
12.6.3. Docker Compose như một dạng IaC . . . 20
12.6.4. Kubernetes — điều phối container cho production . . . 20
12.6.5. Triển khai serverless — khi nào phù hợp? . . . 24
12.6.6. Sidecar Pattern và Service Mesh . . . 25
12.7. Case study: KBM . . . 27
12.7.1. Hiện trạng . . . 27
12.7.2. Phân tích theo mức độ trưởng thành triển khai . . . 28
12.7.3. Phân tích bối cảnh nghiệp vụ . . . 28
12.8. Tổng kết . . . 30
12.9. Đọc thêm . . . 31
Tài liệu tham khảo . . . 31

12

CHƯƠNG 12

Triển khai & DevOps

> Nếu triển khai thấy đau, hãy triển khai thường xuyên hơn; tự động hóa biến một việc đáng sợ thành việc bình thường.
> — Theo tinh thần Jez Humble & David Farley, *Continuous Delivery* (2010)

MỤC TIÊU HỌC TẬP

- Hiểu vì sao triển khai microservices phức tạp hơn monolith và cần **tự động hóa**
- Nắm vững **container hóa** (containerization) với Docker — từ Dockerfile đến image registry
- Sử dụng **Docker Compose** để điều phối việc triển khai nhiều service trên một máy
- Thiết kế **pipeline CI/CD** cho microservices — build, kiểm thử và triển khai tự động
- So sánh các **chiến lược triển khai** (deployment strategies) Rolling, Blue/Green, Canary và Shadow
- Hiểu **Infrastructure as Code** (IaC) và vai trò của các nền tảng điều phối triển khai
- Phân tích kiến trúc triển khai của KBM và đề xuất cải thiện

12.1. Thách thức triển khai microservices

Triển khai monolith chỉ cần sao chép một artifact duy nhất, nhưng với microservices, đó là bài toán điều phối hàng chục dịch vụ độc lập. Sự phức tạp này đòi hỏi các luồng tự động hóa chuyên biệt.

12.1.1. Vấn đề: từ "triển khai một tệp WAR" đến "điều phối N service độc lập"

Trong monolith, triển khai nghĩa là: build một artifact (JAR/WAR), sao chép lên máy chủ và khởi động lại. Toàn bộ quy trình từ build, kiểm thử đến triển khai chỉ xoay quanh một tệp duy nhất.

Trong microservices, triển khai mang ý nghĩa khác hẳn. Thay vì đóng gói một ứng dụng duy nhất lên một máy chủ, hệ thống phải quản lý hàng chục artifact riêng lẻ có nhịp phát hành độc lập. Cơ sở dữ liệu cũng vậy: monolith chỉ cần một luồng cập nhật schema thống nhất, còn mỗi microservice sở hữu schema riêng và phải nâng cấp độc lập mà vẫn giữ tương thích với các dịch vụ xung quanh.

Quá trình khởi động và khôi phục hệ thống cũng phức tạp hơn: các tiến trình cần cơ chế chịu lỗi khi phụ thuộc chưa sẵn sàng, và khi có sự cố cần quay lui (rollback), việc đưa một service về phiên bản cũ mà không phá vỡ hợp đồng giao tiếp với các service khác đòi hỏi kỷ luật tương thích ngược chặt chẽ. Ngay cả kiểm thử tích hợp cũng cần dựng kèm các thành phần liên quan hoặc kiểm thử hợp đồng thay thế.

Newman trong [4a, Ch.6] nhấn mạnh: khả năng **triển khai độc lập** (independent deployment) là lợi ích cốt lõi của microservices — nhưng cũng là thách thức lớn. Nếu không tự động hóa, việc triển khai thủ công sẽ trở thành "nỗi ám ảnh chiều thứ Sáu" khi số dịch vụ tăng lên.

Triển khai độc lập

Khả năng triển khai từng service độc lập là lý do cốt lõi để chọn microservices — nhưng không có CI/CD, kỷ luật quản lý phiên bản và hạ tầng bất biến (immutable infrastructure), lợi ích này biến thành gánh nặng. Mỗi lần phát hành thủ công là rủi ro: bỏ sót bước, sai thứ tự migration, thiếu kế hoạch rollback. Tự động hóa DevOps không phải tùy chọn trong microservices — đó là **điều kiện bắt buộc** để lợi ích độc lập trở thành thực tế.

12.1.2. Từ thủ công đến tự động hóa — tư duy DevOps

Theo Mitra [3, Ch.6], DevOps cho microservices có ba nguyên tắc cốt lõi. Nguyên tắc đầu tiên là **Immutable Infrastructure** (hạ tầng bất biến): tuyệt đối không can thiệp trực tiếp vào máy chủ đang chạy mà phải khởi tạo phiên bản mới và loại bỏ phiên bản cũ, nhằm đảm bảo khả năng tái tạo và triệt tiêu sự sai lệch cấu hình (configuration drift). Kế tiếp là **Infrastructure as Code** (IaC), biến mọi thao tác cấu hình máy chủ, mạng hay cơ sở dữ liệu thành mã nguồn lưu trữ trên Git, giúp dễ dàng rà soát và quay lui. Cuối cùng, **Continuous Delivery** (phân phối liên tục) đảm bảo mã nguồn luôn ở trạng thái sẵn sàng phát hành chỉ với một thao tác kích hoạt, qua đó chia nhỏ rủi ro và tăng tốc vòng lặp phản hồi.



Hình 12.1: *Triển khai thủ công và tự động*

Mô hình triển khai tự động giúp loại bỏ các bước thủ công dễ sinh lỗi và giảm rủi ro khi phát hành phần mềm. Quá trình phân phối tự động phản ánh triết lý vận hành cốt lõi của DevOps: biến khó khăn thành động lực tự động hóa (if it hurts, do it more often).

If It Hurts, Do It More Often

Nếu triển khai thấy đau, hãy triển khai thường xuyên hơn. Chỗ đau cho biết cần tự động hóa điều gì.

— *Theo tinh thần Jez Humble & David Farley, Continuous Delivery*

12.2. Containerization với Docker

12.2.1. Vấn đề: “Works on my machine”

Sự không nhất quán giữa môi trường phát triển (máy cá nhân) và môi trường production thường xuyên gây ra lỗi “Works on my machine”. Container hóa giải quyết bài toán này.

12.2.2. Container — cô lập gọn nhẹ

Container (Docker) đóng gói ứng dụng cùng với các thư viện phụ thuộc (dependencies) và môi trường thực thi (runtime) thành một **image** — chạy giống nhau trên mọi môi trường. Khác với máy ảo (Virtual Machine), container chia sẻ kernel với hệ điều hành máy chủ nên giảm đáng kể dung lượng và khởi động tính bằng giây thay vì tính bằng phút.



Hình 12.2: So sánh kiến trúc máy ảo và container

Khác biệt về kiến trúc giữa hai công nghệ này kéo theo khác biệt đáng kể về hiệu năng và cách vận hành. Về thời gian khởi động, máy ảo thường mất hàng phút do phải nạp lại toàn bộ hệ điều hành khách, trong khi container chỉ mất vài giây vì dùng chung kernel với máy chủ. Về tài nguyên, một máy ảo đòi hỏi dung lượng lưu trữ lên đến hàng gigabyte, trong khi container tối ưu hóa đáng kể tài nguyên do chỉ bao gồm ứng dụng và các thư viện cần thiết.

Đổi lại, mức độ cô lập của hai công nghệ khác nhau: máy ảo cô lập ở mức phần cứng nên phù hợp cho môi trường chia sẻ đa khách hàng (multi-tenant), còn container cô lập ở mức tiến trình của hệ điều hành.

Phòng thí nghiệm riêng biệt và buồng thực hành dùng chung hạ tầng

Để hình dung sự khác biệt về bản chất trong bối cảnh trường đại học:

- **Máy ảo (Virtual Machine)**: Giống như xây cho mỗi nhóm nghiên cứu một phòng thí nghiệm độc lập với hệ thống điện dự phòng, điều hòa và tủ thiết bị riêng hoàn toàn (hệ điều hành khách riêng). Mức cô lập rất cao, nhưng tốn diện tích và mất nhiều thời gian chuẩn bị trước mỗi ca học.
- **Container**: Giống như bố trí cho mỗi nhóm một buồng thực hành cô lập bên trong Trung tâm Thí nghiệm chung: mỗi nhóm có bộ dụng cụ và không gian thao tác riêng (user space), nhưng dùng chung điện lưới, mạng nội bộ và hệ thống thông gió của tòa nhà (OS kernel). Tài nguyên được tận dụng tối ưu và buồng thực hành sẵn sàng chỉ sau vài giây.

Richardson trong [2a, Ch.12] mô tả bốn mô hình triển khai (deployment pattern), trong đó **Service as Container** là phổ biến nhất cho microservices — cân bằng giữa mức độ cô lập, tốc độ khởi tạo và hiệu suất sử dụng tài nguyên.

12.2.3. Các mô hình triển khai — từ đặc thù ngôn ngữ đến Serverless

Bốn mô hình triển khai của Richardson trải dài từ đơn giản đến phức tạp. Cách tiếp cận nguyên sơ nhất là đóng gói ứng dụng theo định dạng đặc thù của ngôn ngữ (như JAR, WAR) rồi triển khai trực tiếp lên application server; dù đơn giản nhưng lại tiềm ẩn nguy cơ cao xảy ra lỗi xung đột thư viện do thiếu tính cô lập. Kế tiếp, việc đặt mỗi dịch vụ lên một máy ảo riêng giải quyết được bài toán cô lập, nhưng tốn nhiều tài nguyên và khởi tạo chậm.

Mô hình mỗi dịch vụ một container (như Docker) cân bằng được tính mỏng nhẹ, khả năng di động và sự độc lập, dù vẫn cần công cụ điều phối ở quy mô lớn. Khi cần tối giản vận hành, kiến trúc Serverless giao toàn bộ hạ tầng cho đám mây để đạt khả năng mở rộng tức thì, chấp nhận độ trễ khởi động và sự phụ thuộc nhà cung cấp. Với các hệ thống lớn, mô hình container thường chạy trên Kubernetes, nền tảng điều phối và tự phục hồi tiêu chuẩn, dù đòi hỏi nhiều công sức tiếp cận ban đầu.

12.2.4. Dockerfile — định nghĩa container image

Dockerfile mô tả cách đóng gói container image cho một service. Mẫu phổ biến cho microservice viết bằng Spring Boot:

Listing 12.1: *Dockerfile sử dụng multi-stage build*

```
# Multi-stage build — tách build environment khỏi runtime
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY pom.xml mvnw ./
COPY .mvn .mvn
RUN ./mvnw dependency:go-offline -B    # Cache dependencies layer
COPY src src
RUN ./mvnw package -DskipTests        # Build JAR

# Runtime image nhỏ hơn JDK
FROM eclipse-temurin:21-jre
WORKDIR /app

# Tạo non-root user (giải pháp khuyến nghị về bảo mật)
RUN useradd -m appuser
USER appuser

COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
# Khởi động ứng dụng (Spring Boot tự quản lý Graceful Shutdown qua cấu hình)
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Hai kỹ thuật quan trọng:

- **Build đa giai đoạn (multi-stage build)**: giai đoạn 1 (JDK) biên dịch tệp JAR, giai đoạn 2 (JRE) chỉ chứa môi trường thực thi, giúp giảm đáng kể dung lượng image
- **Cache theo lớp (layer caching)**: sao chép tệp cấu hình trước mã nguồn (`COPY pom.xml` trước `COPY src`), nhờ đó các thư viện phụ thuộc chỉ được tải lại khi cấu hình thay đổi, thay vì mỗi lần cập nhật mã

Mỗi service một container

Mỗi microservice chạy trong một container riêng. Không đóng gói nhiều dịch vụ vào chung một container — cách làm đó đánh mất tính cô lập, khó mở rộng và khó gỡ lỗi.

— Tổng hợp nguyên tắc container hóa từ Richardson [2a, Ch.12] và Newman [4a, Ch.6]

12.3. Docker Compose — điều phối trên một máy

12.3.1. Vấn đề: khởi chạy thủ công nhiều dịch vụ, cơ sở dữ liệu và Kafka

Nền tảng KBM yêu cầu khởi động nhiều dịch vụ, cơ sở dữ liệu và message broker theo đúng thứ tự. Việc chạy thủ công từng thành phần làm tốn thời gian và dễ dẫn đến sai lệch cấu hình mạng.

Listing 12.2: Khởi động thủ công các service (anti-pattern)

```
# SAI: khởi động thủ công bằng 10+ lệnh docker run,
# đúng thứ tự, đúng network, đúng env vars
docker run -d postgres:15 ...
docker run -d zookeeper:3.8 ...
docker run -d kafka:3.5 ...
docker run -d eureka-server ...
docker run -d kbm-gateway --eureka-url=... ...
docker run -d kbm-core --db-url=... --kafka=... ...
# ... còn 5 services nữa
```

12.3.2. Docker Compose — khai báo nhiều service

Docker Compose định nghĩa **toàn bộ stack** trong một tệp YAML — các service, mạng (network), ổ lưu trữ (volume) và quan hệ phụ thuộc. Một lệnh `docker compose up` khởi động toàn bộ hệ thống.

Cấu trúc Docker Compose cho KBM LMS chính (đơn giản hóa):

Listing 12.3: Cấu hình Docker Compose cho KBM LMS chính

```
# docker-compose.yml — KBM LMS chính
services:
  # --- Infrastructure ---
  postgres:
    image: postgres:15
    restart: unless-stopped
    environment:
      POSTGRES_DB: core_db
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  # Kafka chạy KRaft mode (production-ready từ 3.3, mặc định từ 4.0):
  # broker tự quản metadata, không cần Zookeeper.
  # Cụm dựa trên Zookeeper nay là legacy (gỡ bỏ ở Kafka 4.0).
  kafka:
    image: confluentinc/cp-kafka:7.5.0
    restart: unless-stopped
    environment:
      # bắt buộc ở KRaft: format volume metadata
      CLUSTER_ID: "MkU3OEVBNTcwNTJENDM2Qk"
      KAFKA_PROCESS_ROLES: "broker,controller"
      KAFKA_NODE_ID: 1
      KAFKA_CONTROLLER_QUORUM_VOTERS: "1@kafka:9093"
      KAFKA_LISTENERS: "PLAINTEXT://:9092,CONTROLLER://:9093"
      # Listener tên CONTROLLER không trùng tên security protocol nào, nên phải
      # khai báo map tường minh; thiếu nó broker abort ngay khi validate config.
```

```
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
        "PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT"
      KAFKA_CONTROLLER_LISTENER_NAMES: "CONTROLLER"
      KAFKA_INTER_BROKER_LISTENER_NAME: "PLAINTEXT"
      # Địa chỉ client nhận về sau bootstrap — thiếu thì service không kết nối được
      KAFKA_ADVERTISED_LISTENERS: "PLAINTEXT://kafka:9092"
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1   # single-node
    volumes:
      # giữ log và metadata KRaft qua các lần down/up
      - kafka-data:/var/lib/kafka/data
    healthcheck:
      test: ["CMD", "kafka-cluster", "cluster-id",
             "--bootstrap-server", "localhost:9092"]
      interval: 10s
      timeout: 5s
      retries: 5

  # --- Platform Services ---
  kbm-registry:
    image: ghcr.io/kbm/kbm-registry
    ports: ["9000:9000"]

  kbm-gateway:
    image: ghcr.io/kbm/kbm-gateway
    ports: ["9001:9001"]
    depends_on: [kbm-registry]

  # --- Application Services ---
  kbm-core:
    # nhận tag từ pipeline; thiếu biến này thì luôn kéo latest
    image: ghcr.io/kbm/kbm-core:${IMAGE_TAG:-latest}
    restart: unless-stopped
    depends_on:
      postgres:
        condition: service_healthy
      kafka:
        condition: service_healthy
      kbm-registry:
        condition: service_started
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/core_db
    # --wait ở pipeline chỉ đợi healthy khi có khối này;
    # image cần có wget hoặc curl
    healthcheck:
      test: ["CMD-SHELL",
             "wget -qO- http://localhost:8080/actuator/health || exit 1"]
      interval: 10s
      retries: 12
    deploy:
      resources:
        limits:
          cpus: '1.0'
          memory: 512M

  # kbm-academic, kbm-auth, kbm-notification khai báo tương tự kbm-core
  # (lược bớt cho gọn)
  kbm-judge-sql:
    image: ghcr.io/kbm/kbm-judge-sql
```

```
    depends_on:
      kafka:
        condition: service_healthy
      kbm-registry:
        condition: service_started

volumes:
  pgdata:
  kafka-data:
```

12.3.3. Docker Compose — điểm mạnh và giới hạn

Bốn khía cạnh dưới đây nêu điểm mạnh kèm giới hạn tương ứng của Docker Compose:

- **Về tính khai báo (Declarative)**: Docker Compose định nghĩa toàn bộ stack trong một tệp cấu hình duy nhất, giúp dễ hình dung tổng thể hệ thống; đổi lại, công cụ này *chỉ điều phối trên một máy chủ duy nhất (single host)*.
- **Về tính tái tạo (Reproducible)**: Chỉ với lệnh `docker compose up` là có thể dựng lại môi trường nhất quán trên máy phát triển hay staging. Tuy nhiên, ngoài chính sách khởi động lại tiến trình cơ bản (`restart: unless-stopped`), Compose *không có cơ chế tự phục hồi cấp cụm* (như dời workload sang máy chủ khác khi nút phần cứng gặp sự cố).
- **Về quản lý phụ thuộc (Dependencies)**: Chỉ thị `depends_on` kết hợp `condition: service_healthy` đợi lệnh kiểm tra sức khỏe (ví dụ `pg_isready`) báo thành công trước khi bật container kế tiếp. Dù vậy, lệnh kiểm tra thành công chưa đảm bảo tiến trình bên trong đã hoàn tất mọi bước khởi tạo nghiệp vụ, nên ứng dụng vẫn cần cơ chế thử lại kèm thời gian chờ tăng dần (retry/backoff) khi kết nối tới phụ thuộc. Ngoài ra, Compose *không tích hợp sẵn bộ cân bằng tải nội bộ* giữa nhiều bản sao, đòi hỏi cấu hình thêm reverse proxy nếu muốn nhân bản tiến trình.
- **Về mạng nội bộ và cập nhật (Networking, Updates)**: Mạng nội bộ cho phép các dịch vụ gọi nhau thuận tiện qua tên service. Tuy nhiên, khi cập nhật phiên bản, Compose *thiếu cơ chế cập nhật cuốn chiếu (rolling update) không gián đoạn* dựa trên readiness probe.

Docker Compose **phù hợp cho** môi trường phát triển, staging, kiểm thử tự động và **hệ thống production quy mô nhỏ và vừa** (như KBM LMS chính). Phân hệ DevOps Lab là ngoại lệ có chủ đích: phân hệ này dùng cụm k3s nhỏ kèm Sysbox để cô lập không gian thực hành của sinh viên, chứ không phải vì toàn bộ hệ thống đã đòi hỏi hạ tầng Kubernetes quy mô lớn.

KBM: Docker Compose đủ cho production quy mô nhỏ–vừa

KBM LMS chính dùng Docker Compose thay vì Kubernetes — đây là lựa chọn **hợp lý cho giai đoạn hiện tại**. Docker Compose giảm đáng kể chi phí vận hành (không quản lý etcd, kube-apiserver, node pool) và đủ cho việc triển khai trên một máy chủ với tải vừa phải. Chọn Kubernetes trước khi thực sự cần mở rộng đa máy chủ hay tự phục hồi cấp cụm là thiết kế quá mức cần thiết — không phải giải pháp khuyến nghị.

12.4. Pipeline CI/CD cho microservices

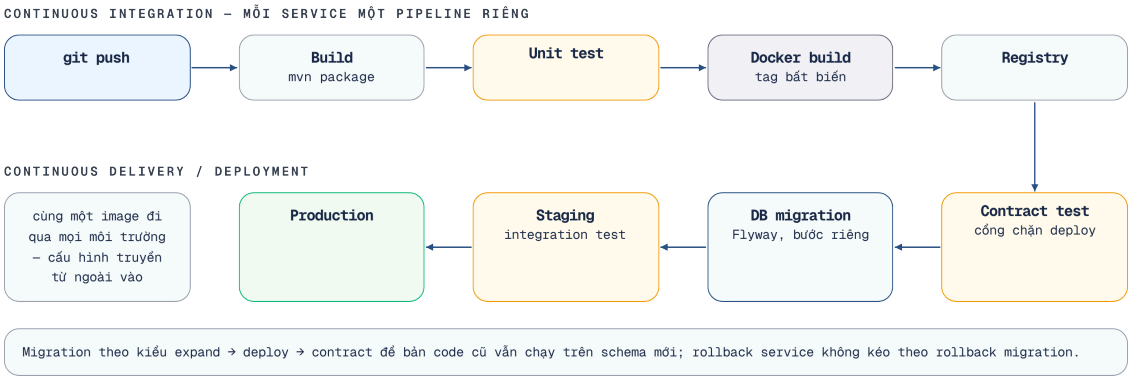
12.4.1. Vấn đề: N service × M giai đoạn = độ phức tạp nhân bội

Trong kiến trúc monolith, pipeline CI/CD thường chỉ là một luồng duy nhất. Với microservices, mỗi dịch vụ sở hữu một vòng đời riêng, đòi hỏi pipeline tự động hóa phải xử lý được các ràng buộc phức tạp:

- Việc triển khai phiên bản mới của Service A **không được phép làm gián đoạn** Service B (API versioning, Chương 3)
- Database migration chạy **trước** bước triển khai service mới
- Rollback service **không** được kéo theo rollback migration: schema phải tương thích ngược (expand → deploy → contract, xem mục Database Migration) để phiên bản mã cũ vẫn chạy được trên schema đã mở rộng — down migration chỉ dùng khi đã chứng minh được tính khả đảo và có bản sao lưu

12.4.2. Kiến trúc CI/CD cho microservices

Hình 12.3 là pipeline CI/CD chuẩn cho microservices, đáp ứng các yêu cầu vừa nêu.



Hình 12.3: *Pipeline CI/CD chuẩn cho microservices*

Pipeline CI/CD và đồ án tốt nghiệp

Toàn bộ pipeline CI/CD có thể ví như quá trình nộp đồ án:

- **Build**: Đóng quyển đồ án và dán mã số sinh viên bất biến (tương tự image tag).
- **Kiểm thử tự động**: Phòng Đào tạo tự động chạy hệ thống quét đạo văn và kiểm tra thể thức văn bản.
- **Deploy Staging**: Sinh viên bảo vệ thử đồ án trước Hội đồng Bộ môn.
- **Deploy Production**: Sau khi duyệt, quyển đồ án chính thức được đưa lên kệ Thư viện trường.

Đặc biệt, tính **bất biến** (immutable) nghĩa là: khi phát hiện lỗi, sinh viên *không được chỉnh sửa trực tiếp trên bản đồ án đã nộp lưu chiểu* (không SSH vào máy chủ sửa mã). Mọi thay đổi phải đi từ bản nháp, đóng quyển mới và làm lại toàn bộ quy trình nộp duyệt.

12.4.3. Mono-repo và Poly-repo — ảnh hưởng đến CI/CD

Câu hỏi đầu tiên khi thiết kế CI/CD là lưu trữ mã nguồn ở đâu. Giới công nghệ chia làm hai trường phái: **Mono-repo** (như cách Google, Meta làm) tích hợp tất cả dịch vụ vào chung một kho lưu trữ, giúp chia sẻ mã nguồn và thực hiện các thay đổi xuyên service rất tiện lợi. Đổi lại, pipeline CI/CD cần bộ lọc đường dẫn để chỉ build những dịch vụ thực sự có thay đổi. Ngược lại, **Poly-repo** (được Netflix, Amazon ưa chuộng) phân tách mỗi dịch vụ thành một kho lưu trữ độc lập: thiết lập N pipeline độc lập rất tự nhiên, nhưng việc chia sẻ thư viện nội bộ tốn thêm bước phát hành phiên bản qua kho gói (artifact repository).

Mitra trong [3, Ch.6] khuyến nghị: đối với đội ngũ quy mô vừa và nhỏ, **mono-repo là lựa chọn đơn giản hơn** — giúp tránh chi phí quản lý (overhead) cho nhiều kho lưu trữ và luồng pipeline. Khi đội ngũ lớn (>20 lập trình viên), poly-repo mang lại mức độ tự chủ cao hơn cho các nhóm.

12.4.4. Tối ưu hóa pipeline

Các giải pháp khuyến nghị trong việc thiết kế pipeline CI/CD bao gồm:

- **Build một lần, triển khai mọi nơi (build once, deploy everywhere)**: Artifact (ví dụ Docker image) chỉ được build một lần duy nhất, sau đó lần lượt được chuyển tiếp (promote) qua các môi trường staging và production. Cách làm này dùng chung một artifact duy nhất, triệt tiêu rủi ro "thử nghiệm chạy tốt nhưng production lỗi" do mã nguồn bị build lại.
- **Cấu hình bên ngoài (externalized config)**: Toàn bộ cấu hình đặc thù của môi trường (như URL cơ sở dữ liệu, khóa API) phải được truyền vào ứng dụng thông qua biến môi trường. Container image phải luôn đảm bảo tính bất biến (immutability) bất kể nó đang được triển khai ở đâu.
- **Pipeline song song (parallel pipeline)**: Mỗi service phải đóng gói, kiểm thử và phân phối được hoàn toàn độc lập. Việc nâng cấp Service A không được trở thành điểm nghẽn (bottleneck) cản trở quá trình phát hành của Service B.
- **Chạy cập nhật schema cơ sở dữ liệu như một bước riêng**: Công cụ quản lý cập nhật schema như Flyway hay Liquibase cần được thực thi như một bước độc lập trước khi khởi chạy ứng dụng mới. Việc tách thay đổi cấu trúc lưu trữ khỏi thay đổi logic mã nguồn giúp quá trình rollback rõ ràng và an toàn hơn.
- **Cổng kiểm thử hợp đồng (contract test gate)**: Trong hệ sinh thái giao tiếp chéo, pipeline chỉ được phép triển khai khi toàn bộ kiểm thử hợp đồng (contract test) đều đạt. Đây là chốt chặn ngăn các thay đổi phá vỡ tương thích (breaking change) lọt lên production.

Build Once, Deploy Everywhere

Build artifact một lần duy nhất, rồi chuyển tiếp chính artifact đó qua các môi trường (dev → staging → production). Nếu mỗi môi trường build lại một lần, thứ được kiểm thử ở staging không còn là artifact chạy trên production.

— Tổng hợp nguyên tắc CI/CD từ Newman [4a, Ch.6] và Mitra [3, Ch.10]

Đảm bảo tính bất biến của artifact qua các môi trường là cơ sở để xây dựng hệ thống tự động hóa đáng tin cậy.

Triển khai thủ công khó duy trì khi số service tăng

Với N service độc lập, mỗi service có bước cập nhật schema riêng, quản lý phiên bản riêng và kế hoạch rollback riêng — việc triển khai thủ công rất chậm và **khó bảo trì khi mở rộng quy mô**. Mỗi thao tác thủ công đều tiềm ẩn rủi ro: sai thứ tự cập nhật schema dữ liệu, thiếu kịch bản rollback, hay lỗi do con người. Tự động hóa CI/CD biến việc triển khai thành một quy trình thường quy, và nghiên cứu DORA cho thấy triển khai thường xuyên hơn lại **giúp giảm tỷ lệ lỗi** (xem Chương 1, DORA Metrics).

12.4.5. Ví dụ thực tế: pipeline CI/CD cho KBM

KBM thực tế tách mỗi dịch vụ thành một kho lưu trữ độc lập (poly-repo) và vận hành trên **GitLab CI** kèm GitLab Container Registry. Để minh họa kỹ thuật lọc theo đường dẫn (`paths` filter) trên một

nền tảng quen thuộc với đa số người đọc, ví dụ dưới đây giả định cấu trúc mono-repo trên **GitHub Actions** cho `kbm-academic` — các bước (build → test → scan → push → deploy) ánh xạ 1:1 sang `.gitlab-ci.yml` (stages/jobs). Mỗi service có workflow riêng và chỉ kích hoạt khi có thay đổi trong thư mục của service đó.

Listing 12.4: *Pipeline GitHub Actions cho `kbm-academic` — build → test → scan → push → deploy*

```yaml
# .github/workflows/kbm-academic.yml
name: kbm-academic CI/CD

on:
  push:
    branches: [main]
    paths: [kbm-academic/**]   # chỉ trigger khi service này thay đổi

permissions:
  contents: read
  # GITHUB_TOKEN mặc định chỉ đọc; cần quyền này để push lên ghcr.io
  packages: write

env:
  REGISTRY: ghcr.io
  IMAGE: ${{ github.repository }}/kbm-academic

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: 'maven'

      - name: Build and test
        run: mvn -f kbm-academic/pom.xml verify

      - name: Log in to registry
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}   # tự động từ GitHub Actions

      - name: Build image (local)
        uses: docker/build-push-action@v5
        with:
          context: kbm-academic
          load: true
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE }}:${{ github.sha }}

      - name: Scan vulnerability
        # ghim theo commit, không dùng nhánh
        uses: aquasecurity/trivy-action@<commit-sha>
        with:
          image-ref: '${{ env.REGISTRY }}/${{ env.IMAGE }}:${{ github.sha }}'
```

```
          format: 'table'
          exit-code: '1'
          severity: 'CRITICAL,HIGH'

      - name: Push image
        run: |
          # Push ĐÚNG image đã scan ở bước trước (không build lại)
          # — giữ nguyên tắc "build once"
          docker tag ${{ env.REGISTRY }}/${{ env.IMAGE }}:${{ github.sha }} \
            ${{ env.REGISTRY }}/${{ env.IMAGE }}:latest
          docker push ${{ env.REGISTRY }}/${{ env.IMAGE }}:${{ github.sha }}
          docker push ${{ env.REGISTRY }}/${{ env.IMAGE }}:latest

  deploy-staging:
    needs: build-test
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - name: Run DB migration
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.STAGING_HOST }}
          username: ${{ secrets.STAGING_USER }}
          key: ${{ secrets.STAGING_SSH_KEY }}
          script: |
            # BẢO MẬT: Inject qua file .env thay vì command line
            # để tránh lộ secret trong `ps aux`
            # dừng ngay khi có lệnh lỗi — exit code thất bại lan về pipeline
            set -Eeuo pipefail
            # luôn dọn file secret, kể cả khi migration thất bại
            trap 'rm -f .env.migration' EXIT
            echo "SPRING_DATASOURCE_URL=${{ secrets.STAGING_DB_URL }}" \
              > .env.migration
            IMAGE_TAG=${{ github.sha }} \
              docker compose -f /app/docker-compose.yml \
              run --rm --no-deps --env-from-file .env.migration kbm-academic \
                --spring.flyway.enabled=true \
                --spring.kafka.listener.auto-startup=false \
                --spring.main.web-application-type=none

      - name: Deploy service
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.STAGING_HOST }}
          username: ${{ secrets.STAGING_USER }}
          key: ${{ secrets.STAGING_SSH_KEY }}
          script: |
            # chặn pipeline đến khi healthcheck báo healthy
            IMAGE_TAG=${{ github.sha }} \
              docker compose -f /app/docker-compose.yml \
              up -d --no-deps --wait kbm-academic

  deploy-production:
    needs: deploy-staging
    runs-on: ubuntu-latest
    environment: production    # yêu cầu manual approval trong GitHub Environments
    steps:
      - name: Run DB migration
```

```
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.PROD_HOST }}
          username: ${{ secrets.PROD_USER }}
          key: ${{ secrets.PROD_SSH_KEY }}
          script: |
            set -Eeuo pipefail
            trap 'rm -f .env.migration' EXIT
            echo "SPRING_DATASOURCE_URL=${{ secrets.PROD_DB_URL }}" \
              > .env.migration
            IMAGE_TAG=${{ github.sha }} \
              docker compose -f /app/docker-compose.yml \
              run --rm --no-deps --env-from-file .env.migration kbm-academic \
                --spring.flyway.enabled=true \
                --spring.kafka.listener.auto-startup=false \
                --spring.main.web-application-type=none

      - name: Deploy to production
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.PROD_HOST }}
          username: ${{ secrets.PROD_USER }}
          key: ${{ secrets.PROD_SSH_KEY }}
          script: |
            IMAGE_TAG=${{ github.sha }} \
              docker compose -f /app/docker-compose.yml \
              up -d --no-deps --wait kbm-academic
```

Một số điểm cần chú ý:

- `paths: [kbm-academic/**]` — trong cấu trúc mono-repo, bộ lọc đường dẫn (path filter) giúp tránh build lại toàn bộ kho mã khi chỉ một dịch vụ thay đổi (với cấu trúc poly-repo như KBM, mỗi kho lưu trữ vốn đã kích hoạt pipeline riêng).
- Bước cập nhật schema (`Run DB migration`) chạy **trước** khi triển khai service ở cả `staging` lẫn `production`, và chạy qua `docker compose ... run --rm --no-deps` để vào đúng mạng nội bộ của cơ sở dữ liệu. Thứ tự này là bắt buộc — xem mục Database Migration trong pipeline CI/CD bên dưới.
- `environment: production` kết hợp GitHub Environments cho phép đặt người duyệt bắt buộc (required reviewers): mỗi lần triển khai lên production cần con người phê duyệt.
- `github.sha` làm image tag thay vì `latest` — đảm bảo nguyên tắc "build once, deploy everywhere": cùng một artifact được chuyển tiếp từ staging lên production, không build lại.

12.4.6. Quản lý secret

Workflow trên dùng nhiều secret (`STAGING_HOST`, `PROD_SSH_KEY`, `STAGING_DB_URL`...). Nguyên tắc cơ bản:

> ⚠ **Không gắn cứng thông tin xác thực**
>
> Đặt thông tin xác thực (credentials) trực tiếp trong tệp `.yml` là lỗi nghiêm trọng — tệp workflow được lưu vào lịch sử Git và có thể bị lộ công khai. Dùng GitHub Secrets (Settings → Secrets and variables → Actions) để lưu trữ. `${{ secrets.GITHUB_TOKEN }}` là token tự động — không cần tạo thủ công.

Cụ thể, các secret thiết yếu của pipeline ví dụ ở trên được phân theo phạm vi (scope) như sau:

- `GITHUB_TOKEN`: Dùng để tự động xác thực với registry. GitHub Actions tự sinh token này và quản lý vòng đời của nó ở phạm vi Repository, không cần thao tác thủ công.

- `STAGING_SSH_KEY` và `PROD_SSH_KEY`: Là các khóa riêng tư SSH dùng để kết nối vào các máy chủ tương ứng. Chúng được giới hạn quyền truy cập ở phạm vi Environment (lần lượt cho môi trường staging và production).
- `STAGING_DB_URL` và `PROD_DB_URL`: Lưu trữ chuỗi kết nối (JDBC URL) đến các cụm cơ sở dữ liệu. Các thông số này cũng được cấu hình nghiêm ngặt theo từng phạm vi Environment tương ứng để loại trừ rủi ro truy cập chéo giữa các môi trường.

Lưu ý: khi cần cô lập, gắn secret với **Environment** (staging/production) thay vì ở mức Repository; khi đó secret của staging không thể dùng trong job của production và ngược lại. Đây là cách GitHub Environments đảm bảo cô lập giữa các môi trường triển khai.

12.4.7. Database Migration trong pipeline CI/CD

Trong microservices, thay đổi schema và thay đổi mã nguồn là hai artifact độc lập — nhưng phải được phối hợp theo thứ tự chính xác. Sai thứ tự dẫn đến gián đoạn dịch vụ hoặc hỏng dữ liệu.

Vấn đề cốt lõi: mã cũ đọc schema mới

Trong quá trình cập nhật cuốn chiếu, luôn tồn tại một khoảng thời gian mà **cả phiên bản cũ và mới** của service cùng hoạt động. Nếu migration tạo schema không tương thích ngược:

- Dịch vụ cũ v1 (đang xử lý request) đọc cột vừa bị đổi tên sẽ dẫn đến lỗi.
- Dịch vụ mới v2 (vừa được triển khai) đọc cột mới sẽ hoạt động bình thường.
- Kết quả: một phần request thành công, một phần lỗi — khó gỡ lỗi, khó quay lui.

Mẫu Expand/Contract — migration tương thích ngược

Giải pháp chuẩn mực cho bài toán này là chia nhỏ thay đổi schema thành chuỗi bước tương thích ngược theo mẫu Expand/Contract. Trước hết ở bước **Expand & ghi song song hai cột**, chúng ta thêm cột/bảng mới cho phép `NULL` mà không đụng tới cột cũ, đồng thời triển khai phiên bản chuyển tiếp ghi đồng bộ vào *cả cột cũ lẫn cột mới* (đường đọc dùng `COALESCE(cột_mới, cột_cũ)`). Kế đến ở bước **Bù dữ liệu lịch sử** (backfill), tác vụ nền chép dữ liệu theo từng lô nhỏ cho các bản ghi cũ còn `NULL` ở cột mới — vì bước ghi song song đã chạy trước, không có bản ghi phát sinh mới nào bị rơi rớt trong lúc backfill. Sau khi toàn bộ dữ liệu đã đồng bộ, ứng dụng chuyển sang đọc/ghi thuần cột mới và ngừng hoàn toàn phiên bản cũ; cuối cùng mới thực hiện bước **Contract (Thu hẹp)** để xóa cột cũ.

Ví dụ minh họa trong `kbm-academic`: đổi tên cột `score` thành `score_10` (thang 10):

Listing 12.5: *Trình tự Expand (kèm ghi song song) → Backfill → Switch → Contract khi đổi tên cột*

```
-- Bước 1 (Expand): thêm cột mới cho phép NULL, GIỮ cột cũ (score)
ALTER TABLE grades ADD COLUMN score_10 DECIMAL(4,2);
-- Triển khai bản chuyển tiếp v1.1: ghi đồng thời cả score và score_10;
-- đường đọc dùng COALESCE(score_10, score) (hoặc gắn DB trigger đồng bộ hai chiều).

-- Bước 2 (Backfill): KHÔNG chạy UPDATE toàn bảng đồng bộ trong migration CI/CD
-- (khóa bảng lớn sẽ treo pipeline và phá vỡ triển khai không gián đoạn).
-- Chạy bằng batch job nền theo từng lô nhỏ cho các dòng lịch sử chưa có score_10:
-- lặp theo lô: ... WHERE id IN (SELECT id FROM grades
--                               WHERE score_10 IS NULL LIMIT 1000)
UPDATE grades SET score_10 = score WHERE score_10 IS NULL;

-- Bước 3 (Switch): khi không còn dòng nào có score_10 IS NULL,
-- triển khai v2 chỉ đọc/ghi score_10 và đợi mọi instance cũ dừng hẳn.
```

```
-- Bước 4 (Contract): xóa cột cũ ở kỳ phát hành kế tiếp
ALTER TABLE grades DROP COLUMN score;
```

Thứ tự thực hiện trong pipeline

⚠ Migration trước khi triển khai

Thứ tự bắt buộc: **migration tương thích ngược → triển khai service mới → (sau khi ổn định) dọn schema cũ**. Nếu triển khai dịch vụ mới trước khi mở rộng schema, ứng dụng mới gặp schema cũ sẽ lỗi ngay khi khởi động hoặc xử lý truy vấn. Flyway và Liquibase kiểm tra checksum và phiên bản schema trước khi service nhận lưu lượng — đây là lý do bước migration trong Listing 12.4 được tách thành một bước riêng thay vì trộn vào quá trình khởi động thông thường.

Trong Listing 12.4, bước migration chạy bằng `docker compose ... run --rm --no-deps` kèm cờ `--spring.main.web-application-type=none`: container tham gia đúng mạng nội bộ để kết nối tới PostgreSQL, thực thi Flyway rồi thoát mà không mở cổng HTTP. Nếu service còn luồng nền, như Kafka listener, phải tắt chúng cho lần chạy này, hoặc dùng image Flyway riêng cho bước migration. Chỉ khi bước này trả mã thoát `0`, pipeline mới cập nhật container dịch vụ thật. Nếu tập lệnh SQL gặp lỗi, `set -Eeuo pipefail` dừng ngay pipeline để kỹ sư kiểm tra thay vì đẩy bản mã mới lên một schema dở dang.

12.5. Các chiến lược triển khai

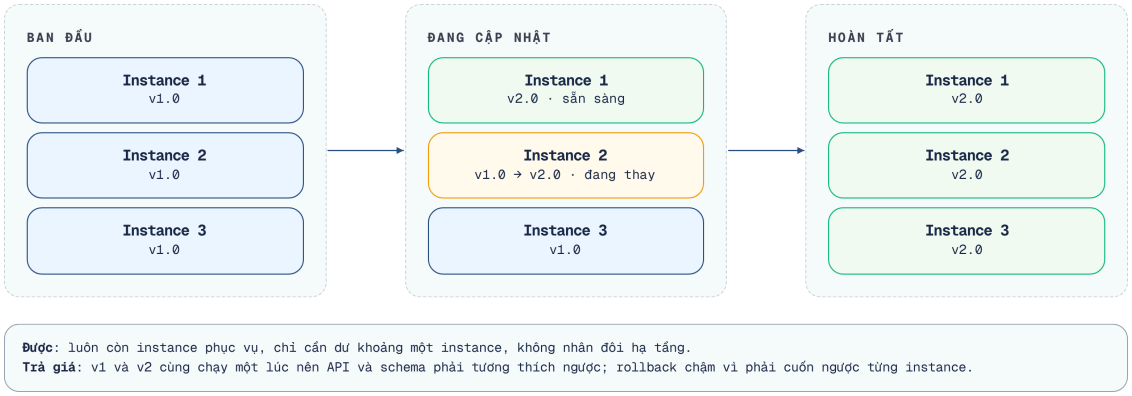
12.5.1. Vấn đề: triển khai phiên bản mới mà không gây gián đoạn

Khi triển khai phiên bản mới của một service, làm thế nào để đảm bảo người dùng không bị gián đoạn? Nếu phiên bản mới tồn tại lỗi, làm thế nào để khôi phục hệ thống nhanh chóng?

Bốn chiến lược được trình bày dưới đây: ba cách phát hành (Rolling, Blue/Green, Canary) và một kỹ thuật kiểm chứng trước khi phát hành (Shadow).

12.5.2. Rolling Update

Thay thế lần lượt **từng instance** — chuyển đổi dần dần từ phiên bản cũ sang phiên bản mới mà không yêu cầu tăng gấp đôi tài nguyên hạ tầng.



Hình 12.4: *Quá trình Rolling Update*

12.5.3. Blue/Green Deployment

Chạy **hai bản hoàn chỉnh** song song — "Blue" (hiện tại) và "Green" (mới). Router chuyển traffic một lần. Để rollback, chỉ cần chuyển router ngược lại; dữ liệu Green đã ghi không tự quay lui, nên Blue phải còn tương thích với schema và dữ liệu mới. Trong Kubernetes, mô hình này thường được thực hiện bằng cách thay đổi `selector` của `Service` để trỏ từ phiên bản Blue sang Green. Quá trình này diễn ra tức thời mà không cần cấu hình tỷ lệ traffic.

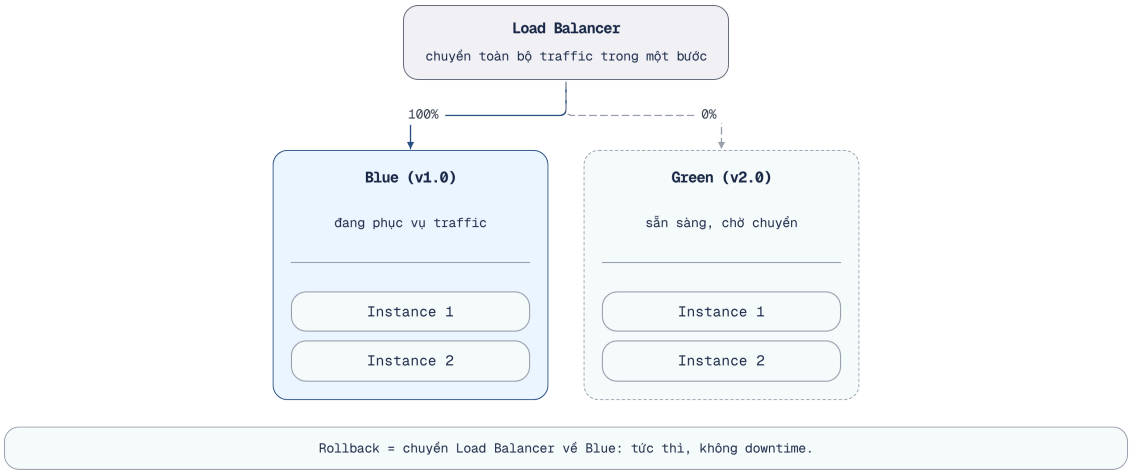
Listing 12.6: *Chuyển traffic lập tức bằng cách patch Service trong Blue/Green*

```
# Triển khai phiên bản Green (đứng chờ, chưa có traffic)
kubectl apply -f deployment-green.yaml

# Khi test trên Green thành công, tiến hành switch traffic lập tức
# bằng cách patch Service
kubectl patch service kbm-academic-svc \
  -p '{"spec":{"selector":{"version":"green"}}}'

# Rollback ngay lập tức nếu có lỗi:
kubectl patch service kbm-academic-svc -p '{"spec":{"selector":{"version":"blue"}}}'
```

Việc thay đổi luồng định tuyến (routing) ở cấp độ mạng giúp hệ thống ngay lập tức phục vụ phiên bản mới mà không tốn thời gian khởi động hoặc làm gián đoạn các kết nối hiện tại.



Hình 12.5: *Kiến trúc Blue/Green Deployment*

12.5.4. Canary Release

Triển khai phiên bản mới cho **một phần nhỏ lưu lượng truy cập**, sau đó giám sát các chỉ số đo lường (metric) và tiến hành mở rộng dần nếu hệ thống hoạt động ổn định. Trong môi trường Kubernetes sử dụng Ingress NGINX, chiến lược Canary có thể được thực hiện bằng cách cấu hình annotation điều hướng một tỷ lệ phần trăm lưu lượng (ví dụ 10%) sang phiên bản mới trong khi 90% vẫn vào phiên bản ổn định.

Listing 12.7: *Cấu hình Canary Ingress phân luồng 10% traffic*

```
# canary-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
```

```yaml
metadata:
  name: kbm-academic-canary-ingress
  annotations:
    nginx.ingress.kubernetes.io/canary: "true"
    # Định tuyến 10% traffic vào Canary
    nginx.ingress.kubernetes.io/canary-weight: "10"
spec:
  rules:
  - host: api.example.edu
    http:
      paths:
      - path: /academic(/|$)(.*)
        pathType: ImplementationSpecific
        backend:
          service:
            name: kbm-academic-canary-svc
            port:
              number: 80
```

Listing này dùng annotation của Ingress NGINX để minh họa cơ chế chia lưu lượng theo trọng số. Dự án Ingress NGINX đã ngừng bảo trì từ tháng 3/2026, nên hệ thống mới nên dùng Gateway API, với trọng số khai báo trong `HTTPRoute`.

12.5.5. Shadow Deployment

Gửi **một bản sao** của traffic thật sang phiên bản mới, trong khi người dùng vẫn chỉ nhận kết quả từ phiên bản cũ. Phiên bản mới xử lý request thật nhưng phản hồi của nó bị loại bỏ, nên lỗi ở phiên bản này không ảnh hưởng tới người dùng. Kỹ thuật này còn được gọi là *traffic mirroring* hoặc *dark launching*.

Điểm mạnh của kỹ thuật này là trả lời được câu hỏi mà staging không trả lời được: phiên bản mới có ổn định dưới tải *thật* hay không. Đổi lại, đây là chiến lược có chi phí cao nhất trong bốn chiến lược: phải chạy song song đủ tài nguyên cho toàn bộ traffic, và mọi **tác dụng phụ** (side effect) phải được cô lập. Nếu bản shadow vẫn ghi vào cùng database hay vẫn gửi email, hậu quả sẽ nhân đôi thay vì chỉ được mô phỏng.

Với KBM, ứng dụng hợp lý nhất là bộ chấm bài: cho `kbm-judge-sql` phiên bản mới chấm song song cùng bộ bài nộp, so kết quả với bản cũ, nhưng chỉ bản cũ mới được ghi điểm cho sinh viên.

12.5.6. So sánh

So sánh bốn chiến lược triển khai: **Rolling Update** giúp đạt zero-downtime với chi phí tài nguyên cực thấp (chỉ cần dư một instance) nhưng có nhược điểm là quá trình rollback chậm. **Blue/Green** cung cấp khả năng rollback tức thời chỉ thông qua việc điều hướng lại router, phù hợp cho các đợt phát hành rủi ro cao nhưng yêu cầu chi phí vận hành gấp đôi. **Canary Release** có cơ chế rollback tức thời và chi phí tài nguyên vừa phải, đồng thời cho phép thử nghiệm trên một nhóm nhỏ người dùng trước khi mở rộng (cơ chế chia lưu lượng giống A/B testing, nhưng mục đích là giảm rủi ro).

Shadow Deployment không chuyển người dùng sang bản mới, nên nó không phải là một cách phát hành mà là một cách *kiểm chứng trước khi* phát hành, và thường đứng trước một trong ba chiến lược kia.

Newman trong [4a, Ch.6] khuyến nghị: chọn chiến lược dựa trên **mức độ rủi ro** của thay đổi. Các bản vá lỗi nhỏ gọn nên áp dụng Rolling Update. Các thay đổi lớn có tầm ảnh hưởng diện rộng cần sử dụng Blue/Green. Những tính năng mới mang tính thử nghiệm nên được triển khai theo mô

hình Canary. Khi rủi ro nằm ở *hiệu năng dưới tải thật* chứ không ở tính đúng đắn của logic, Shadow Deployment là cách đo trước mà không bắt người dùng gánh rủi ro.

12.6. Infrastructure as Code và nền tảng điều phối triển khai

Việc cấu hình máy chủ thủ công qua SSH dễ gây ra tình trạng sai lệch cấu hình giữa các môi trường, khiến chúng không thể tái tạo được (reproducible). Infrastructure as Code (IaC) và các nền tảng điều phối container giải quyết tận gốc bài toán này.

12.6.1. Vấn đề: "Ai cấu hình máy chủ? Cấu hình nằm ở đâu?"

Khi hệ thống microservices chạy trên nhiều máy chủ và container, cấu hình thủ công dẫn đến sai lệch cấu hình nghiêm trọng — một máy chủ có thể bị thay đổi mà không được đồng bộ với máy chủ khác. Tình trạng này khiến môi trường không thể được tái tạo chính xác vì không ai nhớ rõ các bước thiết lập ban đầu. Khi xảy ra lỗi, hệ thống mất khả năng quay lui vì không lưu giữ được trạng thái cấu hình ổn định trước đó.

12.6.2. IaC — hạ tầng dưới dạng mã nguồn

Infrastructure as Code (IaC) nghĩa là mọi thành phần hạ tầng — máy chủ, mạng, cơ sở dữ liệu, message broker — đều được **định nghĩa bằng mã** (thường ở dạng khai báo YAML/HCL), được quản lý phiên bản và áp dụng tự động.

Bên cạnh pipeline CI/CD truyền thống, hệ sinh thái cloud-native mở rộng sang ba hướng bổ trợ:

- **GitOps (Argo CD, Flux)**: Lấy kho Git làm nguồn sự thật duy nhất (single source of truth) và tự động đồng bộ trạng thái hạ tầng, thay cho việc áp cấu hình thủ công.
- **Multi-cluster (Rancher, Karmada)**: Quản lý nhiều cụm ở nhiều vùng cho các tổ chức quy mô toàn cầu, đổi lại là bài toán kết nối mạng liên cụm.
- **Supply chain security**: Các khung kiểm định như SLSA và công cụ ký artifact như Sigstore, đi kèm SBOM (Software Bill of Materials) — bản kê thành phần phần mềm — đang dần trở thành yêu cầu chuẩn.

Các xu hướng này đều *bổ sung* — không thay thế — pipeline CI/CD đã trình bày trong chương này.

Mitra trong [3, Ch.6–7] mô tả IaC là nền tảng cho việc triển khai microservices thông qua một hệ sinh thái công cụ đa dạng, phục vụ cho từng tầng khác nhau của kiến trúc:

- **Tầng Provisioning (cung cấp tài nguyên)**: Bao gồm các công cụ tiêu biểu như Terraform, Pulumi hay CloudFormation. Chúng đảm nhận nhiệm vụ khởi tạo hạ tầng gốc như hệ thống máy ảo (VM), mạng nội bộ (VPC) hay cơ sở dữ liệu dạng dịch vụ được quản lý (managed database).
- **Tầng Container Orchestration (điều phối container)**: Sử dụng các mô tả dạng Kubernetes manifest hay Helm chart để định nghĩa cách thức điều phối ứng dụng, quy tắc scaling và định tuyến lưu lượng.
- **Tầng Configuration Management (quản lý cấu hình)**: Các công cụ như Ansible, Chef hay Puppet hỗ trợ cấu hình hệ điều hành cơ sở và cài đặt thư viện hệ thống mức máy chủ vật lý/ảo.
- **Tầng Service Deployment (triển khai dịch vụ)**: Những nền tảng như Docker Compose, Kubernetes hay Argo CD trực tiếp quản lý vòng đời của các dịch vụ, đảm bảo toàn bộ stack microservices luôn khớp với trạng thái đã khai báo.

Thay vì lặp lại các tệp YAML thô cho từng môi trường, các dự án Kubernetes thực tế thường dùng **Helm** làm trình quản lý gói (package manager) để tham số hóa manifest thành các khuôn mẫu (template) tái sử dụng:

Listing 12.8: Quản lý Kubernetes IaC với Helm

```
# Cài đặt hoặc nâng cấp kbm-academic service với custom values cho production
helm upgrade --install kbm-academic ./kbm-service \
  --namespace kbm-prod \
  --set image.tag=v2.0.1 \
  --set replicaCount=5 \
  --set resources.limits.cpu=1000m
```

12.6.3. Docker Compose như một dạng IaC

Docker Compose — dù gọn nhẹ — đã là một dạng IaC cơ bản: cơ sở dữ liệu, message broker và các service đều được khai báo trong tệp YAML có kiểm soát phiên bản. Nhìn theo lộ trình trưởng thành, các tổ chức thường đi qua ba cấp: **Cấp 1** dùng *Docker Compose* cho môi trường phát triển hoặc triển khai đơn máy chủ (như KBM LMS chính hiện tại); **Cấp 2** dùng *Kubernetes và Helm* khi cần điều phối đa máy chủ, tự phục hồi cấp cụm và tự động mở rộng (DevOps Lab mới áp dụng cục bộ một cụm k3s nhỏ để cô lập bài thực hành); và **Cấp 3** kết hợp *Terraform, Kubernetes và Argo CD* thành mô hình *GitOps* toàn diện.

Infrastructure as Code

Nếu không thể tái tạo toàn bộ hạ tầng từ con số không chỉ bằng mã nguồn, chúng ta chưa thực sự làm chủ hạ tầng đó — mà chỉ đang phụ thuộc vào trí nhớ của người thiết lập nó lần cuối.

— Tổng hợp nguyên tắc IaC từ Mitra [3, Ch.6]

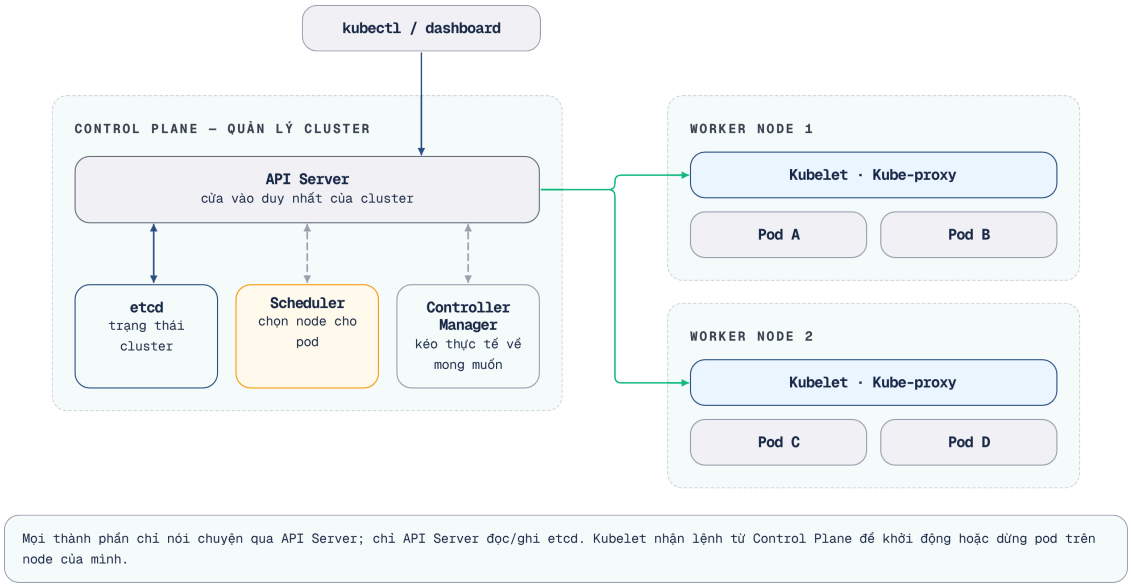
12.6.4. Kubernetes — điều phối container cho production

Phần nâng cao (có thể lướt qua)

Từ đây đến hết mục 12.6 dành cho hệ thống vượt quy mô một máy chủ. Người đọc lần đầu chỉ cần nắm khi nào nên chuyển lên Kubernetes.

Khi hệ thống vượt ngưỡng đơn máy chủ của Docker Compose và đòi hỏi **triển khai đa nút, tự động mở rộng và tự phục hồi**, Kubernetes (K8s) trở thành nền tảng tiêu chuẩn.

Kiến trúc một cụm Kubernetes gồm hai phần chính (Hình 12.6):



Hình 12.6: Kiến trúc Kubernetes — Control Plane và Worker Node

Trong đó, **Control Plane** quản lý toàn bộ cụm: API Server nhận yêu cầu (từ `kubectl` hoặc hệ thống GitOps), etcd lưu trạng thái cụm, Scheduler quyết định pod đặt lên nút nào, còn Controller Manager liên tục đối chiếu để đưa trạng thái thực tế về khớp trạng thái khai báo. Phía **Worker Node** trực tiếp chạy tải công việc (workload): tiến trình Kubelet trên mỗi nút nhận chỉ thị từ Control Plane để khởi tạo hoặc dừng các pod tương ứng.

Các khái niệm cốt lõi

- **Pod**: Đơn vị tính toán nhỏ nhất trong Kubernetes, chứa một hoặc nhiều container dùng chung không gian mạng (network namespace) và ổ lưu trữ gắn kèm.
- **Deployment**: Đối tượng quản lý vòng đời của Pod, duy trì đúng số bản sao (replica) mong muốn và điều khiển cập nhật cuốn chiếu không gián đoạn. Khả năng điều phối động theo trạng thái khai báo này là khác biệt cốt lõi so với Docker Compose.
- **Service**: Cung cấp endpoint mạng ổn định và cân bằng tải nội bộ, che đi việc địa chỉ IP của các Pod thay đổi mỗi lần tái tạo.
- **Ingress**: Bộ định tuyến HTTP/HTTPS ở rìa cụm dựa trên tên miền và đường dẫn URI, thay cho việc cấu hình thủ công từng máy chủ reverse proxy.
- **ConfigMap / Secret**: Tách cấu hình khỏi mã nguồn ứng dụng. Secret API tách dữ liệu nhạy cảm khỏi manifest và kiểm soát truy cập bằng RBAC, nhưng mặc định chỉ encode Base64, chưa phải mã hóa — muốn mã hóa thực sự trên đĩa trong etcd phải bật `encryption at rest` hoặc tích hợp trình quản lý secret bên ngoài. Ở phía Compose, tại thời điểm khảo sát (đầu năm 2026), KBM truyền secret qua các tệp đi cùng triển khai (Chương 9), dù Compose cũng hỗ trợ cơ chế `secrets` gắn vào `/run/secrets`.
- **HPA (Horizontal Pod Autoscaler)**: Tự động tăng/giảm số lượng Pod theo tải thực tế (CPU, bộ nhớ hoặc metric tùy biến).

Dưới đây là tệp YAML hoàn chỉnh cho `kbm-academic` (gồm Deployment, Service, Ingress và HPA), áp dụng cổng chuẩn `:8088` cùng bộ ba *Startup/Liveness/Readiness Probe* và giới hạn tài nguyên (*Resource Limits*):

Listing 12.9: *Cấu hình Kubernetes chuẩn mực cho* `kbm-academic`

```yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kbm-academic
spec:
  # Không khai báo replicas khi đã có HPA bên dưới: mỗi lần apply/GitOps sync
  # sẽ ép số Pod về giá trị cứng này rồi HPA lại kéo về, gây flapping.
  # replicas do HPA sở hữu.
  selector:
    matchLabels:
      app: kbm-academic
  template:
    metadata:
      labels:
        app: kbm-academic
    spec:
      containers:
      - name: kbm-academic
        image: ghcr.io/kbm/kbm-academic:v1.2.0
        ports:
        - containerPort: 8088
        resources:
          requests:
            cpu: "200m"
            memory: "512Mi"
          limits:
            cpu: "500m"
            memory: "1Gi"
        # Bảo vệ JVM trong lúc khởi động nguội (tối đa 12 * 5s = 60s)
        # trước khi bật livenessProbe
        startupProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8088
          periodSeconds: 5
          failureThreshold: 12
        livenessProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8088
          periodSeconds: 10
        # Không có readinessProbe thì kubelet đánh dấu Pod Ready ngay khi container
        # start; Service route traffic vào JVM chưa khởi tạo xong → 502 ở mỗi lần
        # rolling update. Đây chính là cơ chế tạo ra zero-downtime (xem §11.6).
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: 8088
          periodSeconds: 5
          failureThreshold: 3
---
apiVersion: v1
kind: Service
metadata:
  name: kbm-academic-svc
spec:
```

```yaml
  selector:
    app: kbm-academic
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8088
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kbm-academic-ingress
  annotations:
    nginx.ingress.kubernetes.io/use-regex: "true"
    # /$2 giữ lại phần URI sau prefix — rewrite-target: / sẽ cắt cụt mọi path con
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  rules:
  - host: api.example.edu
    http:
      paths:
      - path: /academic(/|$)(.*)
        pathType: ImplementationSpecific
        backend:
          service:
            name: kbm-academic-svc
            port:
              number: 80
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: kbm-academic-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: kbm-academic
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

So sánh chi tiết Docker Compose và Kubernetes

Bảng 12.1 so sánh năng lực của hai nền tảng.

__Bảng 12.1:__ So sánh chi tiết Docker Compose và Kubernetes

Năng lực	Docker Compose	Kubernetes
Multi-host	× Chỉ một máy chủ	✓ Cluster nhiều node
Auto-scaling	× Thủ công `--scale`	✓ HPA dựa trên metric
Self-healing	! `restart: unless-stopped` (cơ bản)	✓ Liveness/readiness probe, tự thay thế
Rolling update	× Gián đoạn khi khởi động lại	✓ Zero-downtime rolling update

Năng lực	Docker Compose	Kubernetes
Service discovery	✓ DNS theo tên service	✓ DNS + load balancing
Secret management	! Có `secrets` gắn vào `/run/secrets`; KBM tại thời điểm khảo sát dùng `.env` và tệp cấu hình	✓ Secret API + RBAC (base64; bật encryption-at-rest để mã hóa thật)
Giới hạn tài nguyên	! Cơ bản (`deploy.resources`)	✓ Request + limit theo từng container
Độ phức tạp thiết lập	Thấp (1 tệp YAML)	Cao (nhiều tệp YAML, thiết lập cluster)
Thời gian học	1–2 ngày	2–4 tuần
Quy mô đội ngũ cần thiết	1–3 người	3–5+ người (hoặc managed K8s)

Kịch bản chuyển đổi của KBM: từ Compose sang Kubernetes

Nếu KBM cần mở rộng hơn (nhiều đơn vị đào tạo, nhiều sinh viên đồng thời):

Listing 12.10: *Kịch bản mở rộng KBM từ Compose lên Kubernetes*

```
Bước 1 (Hiện tại): Docker Compose cho LMS chính
├── Phù hợp hạ tầng nhỏ và workload vừa phải
├── Triển khai: docker compose up -d
└── Chi phí vận hành thấp, thao tác đơn giản

Bước 2 (Mở rộng): Managed Kubernetes (GKE/EKS/AKS)
├── Nhiều đơn vị đào tạo hoặc lưu lượng kỳ thi lớn
├── kbm-judge-sql: HPA mở rộng 1→10 pod trong kỳ thi
├── Core Service: 2-3 replica cho độ sẵn sàng cao
├── PostgreSQL: managed service (Cloud SQL/RDS)
└── Chi phí và độ phức tạp tăng đáng kể
```

Khi nào chuyển lên Kubernetes?

Không chuyển lên K8s vì "Netflix dùng". Chuyển khi có **ít nhất 2 tiêu chí**: (1) cần triển khai đa nút hoặc cô lập workload tốt hơn, (2) cần auto-scaling (tải biến động: kỳ thi → bình thường), (3) cần triển khai zero-downtime (SLA yêu cầu), (4) đội ngũ đủ năng lực vận hành K8s. Managed Kubernetes (GKE, EKS, AKS) giảm đáng kể độ phức tạp — không cần tự dựng/bảo trì control plane.

12.6.5. Triển khai serverless — khi nào phù hợp?

Scale-to-Zero trong Kubernetes

Bên cạnh các dịch vụ serverless thuần túy từ nhà cung cấp đám mây, trên môi trường Kubernetes, xu hướng hiện nay là dùng **KEDA** (Kubernetes Event-driven Autoscaling) hoặc **Knative** để mang lại trải nghiệm tương tự serverless: cho phép giảm số pod xuống 0 khi không có tải (Scale-to-Zero), giúp tiết kiệm đáng kể chi phí cho các dịch vụ chạy nền.

Richardson trong [2a, Ch.12] liệt kê **Serverless** (AWS Lambda, Google Cloud Functions, Azure Functions) là một mô hình triển khai cho microservices. Thay vì quản lý container, chúng ta triển khai các *function* — nhà cung cấp đám mây quản lý hạ tầng, tự động mở rộng và tính phí theo lượt gọi.

Bảng 12.2: So sánh container (Docker/K8s) và serverless

Khía cạnh	Container (Docker/K8s)	Serverless
Quản lý máy chủ	Tự quản lý (hoặc managed K8s)	Nhà cung cấp đám mây quản lý hoàn toàn
Mở rộng	Cấu hình quy tắc auto-scaling	Tự động mở rộng (0 → N)
Mô hình chi phí	Trả theo máy chủ/giờ (dù có chạy hay không)	Trả theo lượt gọi (không dùng = không trả)
Cold start	Thường không (replica đã chạy sẵn); vẫn có khi khởi tạo hoặc scale-out	Có (100 ms–3 s khởi động lần đầu)
Stateful	Có thể (volume, session)	Chỉ stateless
Giới hạn thời gian chạy	Không giới hạn	Thường tối đa 15 phút mỗi lượt gọi
Phụ thuộc nhà cung cấp (vendor lock-in)	Thấp (Docker dễ di chuyển)	Cao (API riêng mỗi đám mây)

Khi nào serverless phù hợp cho microservices?

- **Khối lượng công việc mang tính đột biến và hướng sự kiện (event-driven, bursty workload)**: xử lý tải lên tệp, thay đổi kích thước ảnh, gửi thông báo — không yêu cầu máy chủ duy trì hoạt động liên tục
- **Hàm kết nối (glue function)**: kết nối các service, biến đổi dữ liệu, kích hoạt luồng xử lý
- **Làm bản mẫu (prototype/MVP)**: triển khai nhanh chóng mà không đòi hỏi thiết lập hạ tầng phức tạp

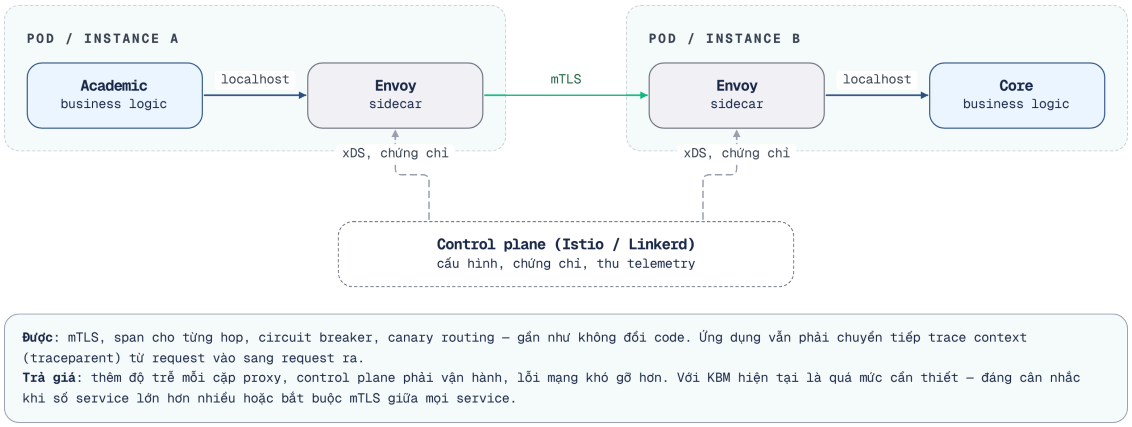
Khi nào không phù hợp?

- **Các luồng xử lý thiết yếu yêu cầu độ trễ thấp**: thời gian khởi động nguội không phù hợp cho các giao tiếp API đồng bộ (luồng nộp bài của LMS)
- **Tiến trình dài hoặc giữ tài nguyên cố định**: `kbm-judge-sql` giữ kết nối tới các sandbox DBMS và mỗi bài chạy 5–30 giây, cộng hàng đợi dồn trong kỳ thi — không hợp mô hình hàm ngắn hạn tính tiền theo lượt gọi
- **Các stateful service**: yêu cầu duy trì kết nối cơ sở dữ liệu hoặc bộ nhớ đệm, trong khi function chỉ tái sử dụng được kết nối lúc môi trường thực thi còn ấm, nên phải tự xử lý kết nối lại, concurrency và giới hạn số kết nối của database

Trong LMS, phần **gửi email/push** khi có sự kiện là ứng viên tốt cho serverless: hướng sự kiện (nhận sự kiện từ Kafka), đột biến theo kỳ thi, không giữ trạng thái. Kênh WebSocket của Notification Service thì không — nó giữ kết nối lâu với trình duyệt. `kbm-core`, `kbm-judge-sql` và `kbm-gateway` phù hợp với container hơn.

12.6.6. Sidecar Pattern và Service Mesh

Khi hệ thống microservices lớn (20+ service), mỗi service cần tự triển khai cùng các nghiệp vụ cắt ngang (cross-cutting concern): mTLS, logging, tracing, circuit breaker, rate limiting. **Sidecar Pattern** giải quyết bằng cách đặt một **tiến trình proxy bên cạnh mỗi instance của service** — proxy xử lý các nghiệp vụ cắt ngang, service chỉ tập trung vào business logic.



Hình 12.7: Sidecar Pattern — proxy xử lý các nghiệp vụ cắt ngang (mTLS, tracing)

Service Mesh (Istio, Linkerd) gồm sidecar proxy trên mọi service và control plane quản lý tập trung. Nó tự động cung cấp mTLS giữa các service, distributed tracing, traffic management (canary routing) và circuit breaking, **gần như không cần thay đổi code** (ngoại lệ quan trọng của tracing được nói rõ bên dưới). Xu hướng gần đây là **sidecarless / ambient mesh** (Istio Ambient, Cilium Service Mesh dựa trên eBPF): mTLS và định tuyến L4 được xử lý ở tầng node/kernel thay vì chạy một proxy riêng cho mỗi pod, nhờ đó cắt bớt chi phí phụ trội (overhead) trên mỗi hop và chi phí vận hành sidecar — đáng cân nhắc nếu đội ngũ đánh giá service mesh cho hệ thống mới.

So với việc cấu hình riêng lẻ trong từng ứng dụng, Service Mesh tập trung hóa các bài toán hạ tầng mạng:

- **Bảo mật luồng giao tiếp (mTLS)**: Thay vì buộc các lập trình viên phải tự triển khai logic mã hóa và quản lý chứng chỉ, các sidecar proxy của Service Mesh tự động thực hiện handshake và thiết lập kết nối mTLS giữa các microservice, trong suốt với ứng dụng.
- **Truy vết phân tán (Tracing)**: Sidecar tự sinh span cho từng hop mạng và export về backend, giảm đáng kể phần hạ tầng phải tự dựng. Tuy nhiên, ứng dụng vẫn **bắt buộc** phải chuyển tiếp trace context (`traceparent` hoặc B3 header) từ request đi vào sang request gửi ra. Proxy không thể tự suy ra outbound request nào bắt nguồn từ inbound request nào; thiếu bước này, các span sẽ không ghép được thành một trace hoàn chỉnh.
- **Cơ chế tự bảo vệ (Circuit Breaker)**: Thay vì phải viết các cấu hình ngắt mạch qua Resilience4j lồng ghép vào business logic, mọi quy tắc đều được khai báo tập trung ở lớp Envoy proxy, qua đó loại bỏ logic xử lý lỗi kết nối khỏi ứng dụng.
- **Định tuyến nâng cao (Canary routing)**: Quá trình phân luồng lưu lượng được thực hiện thông qua các tệp cấu hình khai báo (declarative), thay cho thao tác thủ công trên Load Balancer.
- **Đánh đổi về hiệu năng (Overhead)**: Mặc dù giảm độ phức tạp trong mã nguồn, Service Mesh làm tăng độ trễ mạng do request phải đi qua các lớp proxy. Benchmark chính thức của Istio ghi nhận khoảng 2–3 ms ở P90 cho một cặp proxy; con số thực tế phụ thuộc cấu hình telemetry và tải, nên phải đo trên chính hệ thống của mình.

Với KBM hiện tại, việc áp dụng Service Mesh vẫn mang tính thiết kế quá mức (over-engineering) cho hệ thống LMS chính. Mặc dù việc có thêm DevOps Lab khiến hệ thống trở thành đa ngôn ngữ (polyglot Java+Go), nhu cầu thiết yếu trước mắt vẫn là hoàn thiện CI/CD, tính khả quan sát (observability) và quản lý cấu hình bảo mật. Service Mesh phù hợp khi số lượng service lớn hơn

nhiều, triển khai nhiều máy chủ trở thành mặc định, hoặc yêu cầu bảo mật cao đến mức mTLS bắt buộc giữa mọi service.

⚠ Độ phức tạp khi vận hành và xử lý sự cố

Service Mesh giải quyết nhiều nghiệp vụ cắt ngang, nhưng đổi lại là độ phức tạp vận hành lớn. Vận hành Control Plane (như Istio) và gỡ lỗi mạng ở tầng proxy/eBPF gây khó khăn đáng kể cho việc bảo trì nếu đội ngũ không có chuyên gia mạng. Việc đưa Service Mesh vào khi chưa thực sự cần thiết tiềm ẩn rủi ro gián đoạn dịch vụ.

12.7. Case study: KBM

Phần này đối chiếu các nguyên lý tự động hóa và triển khai với hiện trạng hệ thống KBM, từ đó đề xuất lộ trình nâng cấp phù hợp với quy mô vận hành thực tế.

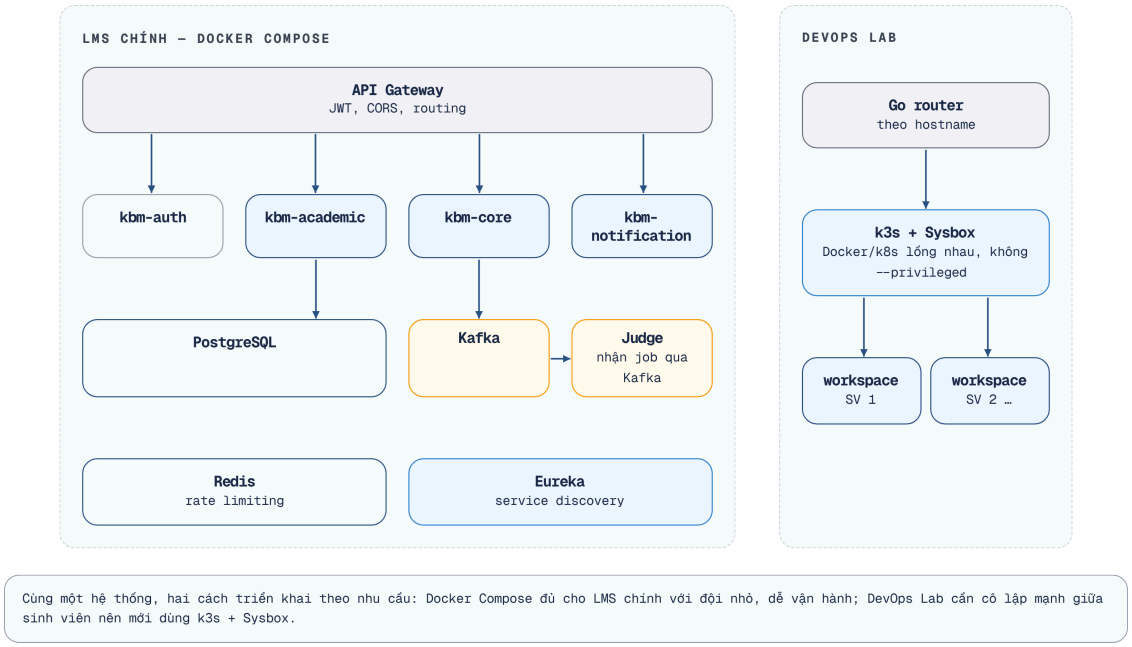
12.7.1. Hiện trạng

KBM hiện có hai kiểu triển khai song song ở mức khái quát:

1. **LMS chính**: các service Java/Spring Boot, database, Kafka, Gateway và frontend triển khai bằng Docker Compose trên hạ tầng nhỏ. Đây là lựa chọn thực dụng cho đội ngũ phát triển quy mô nhỏ, đảm bảo tính dễ vận hành mà không đòi hỏi thiết lập cụm (cluster) phức tạp.
2. **DevOps Lab**: phần thực hành Docker/Kubernetes dùng Go, k3s và Sysbox để tạo môi trường lab cô lập. Đây là nhu cầu đặc thù của miền nghiệp vụ: sinh viên cần chạy container/lab riêng trong môi trường an toàn.

Điều đáng chú ý là **sự đánh đổi trong triển khai**: cùng một hệ thống có thể dùng Docker Compose cho phần LMS chính và k3s/Sysbox cho bounded context cần cô lập.

Bài toán ở DevOps Lab là sinh viên cần chạy Docker/Kubernetes *bên trong* container thực hành. Cách làm phổ biến — Docker-in-Docker (DinD) — đòi hỏi cấp cờ `--privileged`, tức là mở gần như toàn quyền của host cho container, một rủi ro không chấp nhận được trong môi trường nhiều người dùng. Sysbox giải quyết đúng chỗ đánh đổi này: nó cho phép chạy Docker/K8s lồng nhau mà không cần đặc quyền, nhờ tận dụng user namespace và ảo hóa một phần `procfs`/`sysfs`.



Hình 12.8: Kiến trúc triển khai của KBM ở mức khái quát

12.7.2. Phân tích theo mức độ trưởng thành triển khai

Bảng 12.3 đối chiếu hiện trạng triển khai của KBM theo từng khía cạnh kỹ thuật để xác định các điểm nghẽn cần ưu tiên cải thiện.

Bảng 12.3: Phân tích mức độ trưởng thành triển khai của KBM

Khía cạnh	Hiện trạng	Mức trưởng thành	Nhận xét
Containerization	Docker image cho tất cả service	● Tốt	Multi-stage build, container registry (GitLab)
Orchestration	Docker Compose cho LMS chính; k3s/Sysbox cho DevOps Lab	● Phù hợp theo ngữ cảnh	Không ép một nền tảng cho mọi phần
CI/CD	GitLab CI đã phủ các phần chính; một số phần còn build + push thủ công	● Chưa đồng đều	Phần triển khai thủ công còn lại vẫn tạo rủi ro lệch phiên bản (version drift)
Chiến lược triển khai	All-at-once (stop → deploy → start)	● Có downtime	Sinh viên bị gián đoạn khi triển khai
IaC	Các tệp Docker Compose được quản lý phiên bản	● IaC cơ bản	Tái tạo được nhưng thiếu tự động hóa
Quản lý cấu hình	`application.yml` trong container	● Mới tách một phần	Một số cấu hình còn gắn cứng, chưa tách hoàn toàn ra ngoài
Cô lập lab	k3s + Sysbox cho các pod thực hành	● Đúng bài toán	Cần resource quota, RBAC, NetworkPolicy rõ ràng

12.7.3. Phân tích bối cảnh nghiệp vụ

KBM phục vụ sinh viên, do đó khung thời gian triển khai (**deployment window**) đóng vai trò then chốt:

Khi đã có hạ tầng cho nhiều chiến lược (Giai đoạn 3 bên dưới), có thể chọn theo thời điểm. Ngoài cửa sổ đóng băng thay đổi của kỳ thi, các đợt phát hành cần rollback nhanh có thể chọn **Blue/**

Green trong cửa sổ đó chỉ xử lý thay đổi khẩn cấp theo quy trình đã định. Trong những ngày **giữa tuần**, tải trọng trung bình cho phép áp dụng **Rolling Update** nếu đã có orchestrator. Các đợt bảo trì lớn vào kỳ **nghỉ hè** thường không chịu áp lực downtime, nên cách triển khai toàn bộ cùng lúc (**All-at-once**) là hoàn toàn chấp nhận được. Cuối cùng, với những bản vá khẩn cấp (**hotfix**), nên đi qua một đợt **Canary** ngắn (nếu hạ tầng cho phép) để thử nghiệm thận trọng trên một nhóm nhỏ trước khi triển khai trên toàn hệ thống.

CI/CD chưa đồng đều giữa các phân hệ

KBM đã container hóa và vận hành GitLab CI cho các phần chính (§12.4), nhưng một số phần vẫn **triển khai thủ công**: lập trình viên build image trên máy cá nhân, đẩy lên registry, sau đó truy cập từ xa (SSH) vào hạ tầng và thực thi lệnh triển khai. Ở những phần này, kiểm thử và chuyển giao chưa được tự động hóa.

Thực trạng này khiến kiểm thử tự động trước khi triển khai chưa phủ đều, cơ chế rollback chưa nhất quán, và mức độ CI/CD giữa LMS chính với DevOps Lab chênh lệch. Hệ quả là khi có lỗi trên production: (1) lập trình viên phát hiện qua báo cáo của người dùng, (2) quy trình sửa mã nguồn, đóng gói, đẩy lên registry và triển khai lại được thực hiện hoàn toàn thủ công, (3) nếu sửa sai, phải lặp lại toàn bộ quy trình đó. Các yếu tố này làm tăng MTTR (Mean Time To Recovery) và gây ra rủi ro triển khai nhầm phiên bản.

Lộ trình cải thiện có thể chia thành bốn giai đoạn theo thứ tự ưu tiên về chi phí và tác động:

- **Giai đoạn 1 — Phủ đều CI/CD (nỗ lực thấp, tác động cao)**: Mở rộng quy trình GitLab CI hiện có sang toàn bộ service, tự động chạy kiểm thử và build Docker image sau mỗi lần đẩy mã ở mọi phân hệ. Mục tiêu là không còn đoạn mã nào chưa qua kiểm thử lên được production, và mọi phiên bản đều truy vết được.
- **Giai đoạn 2 — Triển khai tự động (nỗ lực trung bình)**: Thiết lập triển khai tự động lên staging, kèm smoke test và một bước xét duyệt thủ công trước khi lên production. Kèm health check cho mỗi dịch vụ, việc triển khai chỉ cần một nút bấm, lịch sử kiểm toán được lưu lại và không ai phải SSH vào máy chủ.
- **Giai đoạn 3 — Triển khai không gián đoạn (zero-downtime) (nỗ lực trung bình–cao)**: Để sinh viên không gặp gián đoạn khi hệ thống cập nhật, dùng Blue/Green sau reverse proxy (hai bộ container, chuyển upstream), vì Compose không có rolling update đúng nghĩa; rolling update chỉ có sau khi chuyển sang orchestrator ở Giai đoạn 4.
- **Giai đoạn 4 — Chuyển sang Kubernetes (nỗ lực cao)**: Chỉ nên thực hiện khi nhu cầu tải thực sự vượt quá khả năng của Docker Compose. Kubernetes mang lại khả năng tự phục hồi, nhưng áp dụng khi chưa cần thiết sẽ phải gánh một khối lượng vận hành không tương xứng.

Rủi ro trong quá trình triển khai

Việc triển khai microservices phức tạp hơn đáng kể so với monolith. Một quyết định sai lầm trong quá trình này có thể làm đình trệ toàn bộ hệ thống hoặc gây ra gián đoạn kéo dài.

1. **Triển khai vào chiều thứ Sáu**: Khi đội ngũ phát hành tính năng mới vào cuối tuần, các lỗi phát sinh thường không được phát hiện do thiếu nhân sự trực. Hậu quả là thời gian gián đoạn có thể kéo dài đến đầu tuần sau. *Phòng tránh*: ưu tiên triển khai vào các buổi sáng giữa tuần khi đội ngũ sẵn sàng xử lý sự cố. Với hệ thống KBM, cần tuyệt đối tránh triển khai trước hoặc trong kỳ thi và buổi thực hành.

2. **Dùng Kubernetes cho hệ thống 3 service**: Tâm lý "Netflix dùng Kubernetes, chúng ta cũng nên" sẽ dẫn đến gánh nặng quản trị phức tạp (cluster management, RBAC, networking) cho một đội ngũ nhỏ. *Phòng tránh*: Docker Compose là đủ cho các hệ thống nhỏ. Việc chuyển đổi sang Kubernetes chỉ thực sự cần thiết khi có nhu cầu chạy trên nhiều máy chủ, yêu cầu mức độ cô lập cao hoặc tự động mở rộng.
3. **Build image khác nhau cho mỗi môi trường**: Việc build riêng cho môi trường phát triển, staging và production sẽ tạo ra các artifact khác nhau. Hậu quả là ứng dụng hoạt động tốt trên staging nhưng lại gặp lỗi trên production. *Phòng tránh*: build một lần, triển khai mọi nơi — đưa toàn bộ cấu hình phụ thuộc môi trường ra ngoài qua biến môi trường.
4. **Thiếu kế hoạch rollback**: Khi phiên bản mới gặp lỗi nhưng không có quy trình rollback sẵn sàng, việc can thiệp thủ công trong tình huống khẩn cấp dễ gây ra lỗi phát sinh nghiêm trọng. *Phòng tránh*: mọi quy trình triển khai phải đi kèm tài liệu hướng dẫn rollback, quy định rõ ràng các bước kỹ thuật để đưa hệ thống về trạng thái ổn định.
5. **Shared database giữa các service**: Chương 7 đã phân tích khía cạnh dữ liệu; ở khía cạnh triển khai, nếu Service A và B dùng chung database, bước cập nhật lược đồ (schema) của A có thể làm hỏng B ngay lúc phát hành. *Phòng tránh*: mỗi service sở hữu database schema riêng để cập nhật schema độc lập.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Các chiến lược triển khai (Blue/Green, Canary)**: deployment-strategies.html
- **Cấu hình bên ngoài (Externalized Config)**: config-server.html

12.8. Tổng kết

Triển khai microservices phức tạp hơn monolith do có nhiều dịch vụ cần được build, kiểm thử, triển khai và khôi phục độc lập nhưng vẫn phải đảm bảo tính tương thích chéo. **Containerization** (Docker) giải quyết vấn đề "works on my machine" — đóng gói service và các thư viện phụ thuộc thành một image có thể chạy ở mọi nơi. **Docker Compose** cho phép điều phối nhiều service trên hạ tầng nhỏ — phù hợp cho đội ngũ quy mô nhỏ và vừa cùng môi trường phát triển/staging.

Pipeline CI/CD là nền tảng cốt lõi: code push → auto build → auto test → auto deploy. Thiếu CI/CD, việc triển khai microservices trở thành một chuỗi thao tác thủ công: chậm, nhiều rủi ro và không tái tạo được (reproducible). Ba chiến lược triển khai chính — Rolling, Blue/Green, Canary — cần được lựa chọn theo mức độ rủi ro: các bản vá lỗi nhỏ áp dụng Rolling Update, thay đổi cấu trúc lớn sử dụng Blue/Green, trong khi các tính năng mới nên được thử nghiệm qua Canary. Shadow là kỹ thuật kiểm chứng trước khi phát hành, dùng khi rủi ro nằm ở hiệu năng dưới tải thật.

Infrastructure as Code đảm bảo hạ tầng tái tạo được và được quản lý phiên bản — tệp Docker Compose đã là IaC cơ bản. Khi mở rộng, Terraform + Kubernetes + Argo CD tạo thành một pipeline GitOps hoàn chỉnh — nhưng cần tránh việc thiết kế quá mức (over-engineering): hãy chọn mức điều phối (orchestration) theo từng bounded context.

Phân tích KBM cho thấy khâu container hóa đã tốt (Docker image, registry) và DevOps Lab đã có nhu cầu k3s/Sysbox riêng, nhưng CI/CD chưa phủ đều và mỗi lần triển khai vẫn gây gián đoạn

(all-at-once). Lộ trình cải thiện: CI/CD cơ bản → triển khai tự động → chiến lược zero-downtime → chuyển sang Kubernetes khi thật sự cần.

Qua 12 chương, chúng ta đã đi từ nền tảng SOA/Microservices (Chương 1–2), qua các mẫu giao tiếp (Chương 3–6), quản lý dữ liệu (Chương 7), hạ tầng (Chương 8–9), chuyển đổi thực tế (Chương 10), observability (Chương 11), đến triển khai (Chương 12). Chuyển đổi từ monolith sang microservices không phải là một lần "Big Bang" mà là **chuỗi quyết định nhỏ, mỗi quyết định mang lại giá trị thực tiễn**, kèm theo hiểu biết về sự đánh đổi (trade-off) của từng mẫu thiết kế.

12.9. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns* 1st Ed. — Ch.12: Deploying Microservices — deployment patterns (language-specific, VM, container, serverless)
2. [4a] Sam Newman, *Building Microservices* — Ch.6: Deployment — CI, build pipelines, CD, Docker, service-to-host mapping và nền tảng điều phối container
3. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.6–7: Infrastructure Pipeline & Infrastructure — IaC, Terraform, Kubernetes; Ch.10: Releasing — Docker, Helm, Argo CD; Ch.11: Managing Change — deployment patterns

Sách bổ trợ:

4. [2b] Chris Richardson, *Microservices Patterns* 2nd Ed. — Ch.18–19: Deploying Microservices & on Kubernetes (updated)
5. [4b] Sam Newman, *Monolith to Microservices* — Ch.3: Splitting the Monolith — deployment considerations during migration

Nguồn trực tuyến:

- Docker official docs — docs.docker.com
- Docker Compose documentation — docs.docker.com/compose
- Kubernetes official docs — kubernetes.io/docs
- Martin Fowler, "Continuous Delivery" — martinfowler.com/bliki/ContinuousDelivery.html
- Martin Fowler, "BlueGreenDeployment" — martinfowler.com/bliki/BlueGreenDeployment.html
- Terraform by HashiCorp — terraform.io
- Argo CD — argo-cd.readthedocs.io (GitOps for Kubernetes)

Bài tập: 12-0, 12-1, 12-2 ở phần Bài tập & case study cuối sách.

Tài liệu tham khảo

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[2b] Richardson, C. (2025). *Microservices Patterns* (2nd ed.; bản MEAP, truy cập 07/2026). Manning Publications.

[3] Mitra, R., & Nadareishvili, I. (2020). *Microservices: Up and Running: A Step-by-Step Guide to Building a Microservices Architecture*. O'Reilly Media.

[4a] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.

[4b] Newman, S. (2019). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media.